SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO5: *Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code.* *(BL3)*

Assessment Tool Weight-age: 40%

Duration :60 Mins
On Class
QUESTIONS: 5 **Total Marks:50**

Annexure A

SIMULATION TASK

Code of Conduct:

I Tejaswi Reddy K (192371036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q. No	Conceptual Understanding (20)	Model Design (25)	Tool Usage(20)	Analysis of Results(20)	Documentation(15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	**/ 25**	**/ 20**	**/ 20**	**/ 15**	**/ 100**

Annexure A

SIMULATION TASK

.Q1. A compiler receives an intermediate representation of an arithmetic expression containing multiple operators and temporary values.

Simulation Tasks

i. Simulate instruction selection for the arithmetic operations.
ii. Allocate registers for operands and temporary values.
iii. Generate the corresponding target instruction sequence.
iv. Show the register contents after each instruction.
v. Evaluate the efficiency of the generated code. (10 Marks)

SOLUTION –

i. Instruction Selection

We select target instructions for each arithmetic operation.

IR Instruction	Selected Target Instructions
`T1 = A + B`	`MOV R1, A` `ADD R1, B`
`T2 = C - D`	`MOV R2, C` `SUB R2, D`
`T3 = T1 * T2`	`MUL R1, R2`
`X = T3 + E`	`ADD R1, E` `MOV X, R1`

The basic idea is to **keep intermediate results in registers** rather than repeatedly storing them in memory.

ii. Register Allocation

Value	Register
`A`	R1
`B`	Used with R1
`T1 = A+B`	R1
`C`	R2
`D`	Used with R2
`T2 = C-D`	R2

`T3 = T1*T2`	R1
`E`	Used with R1
`X`	Memory

Therefore:

```
R1 → T1 → T3 → final result
R2 → T2
```

We don't need a separate register for every temporary because registers can be **reused** once a temporary is no longer needed.

iii. Target Instruction Sequence

The generated target code is:

```
1. MOV R1, A
2. ADD R1, B
3. MOV R2, C
4. SUB R2, D
5. MUL R1, R2
6. ADD R1, E
7. MOV X, R1
```

This assumes a simple two-operand instruction architecture where `ADD R1, B` means R1=R1+BR1 = R1 + B

Similarly `MUL R1, R2` means R1=R1×R2R1 = R1 \times R2

iv. Register Contents After Each Instruction

Initially

```
R1 = empty
R2 = empty
R3 = empty
R4 = empty
```

Instruction 1

```
MOV R1, A
```

Register contents:

```
R1 = A
R2 = -
R3 = -
R4 = -
```

Instruction 2

```
ADD R1, B
```

Now:

R1=A+BR1=A+B

```
R1 = A + B
R2 = -
R3 = -
R4 = -
```

Therefore:

```
T1 = R1 = A + B
```

Instruction 3

```
MOV R2, C
```

Register contents:

```
R1 = A + B
R2 = C
R3 = -
R4 = -
```

Instruction 4

```
SUB R2, D
```

Now:

R2=C−DR2=C-D

```
R1 = A + B
R2 = C - D
R3 = -
R4 = -
```

Therefore:

```
T2 = R2 = C - D
```

Instruction 5

```
MUL R1, R2
```

Now:

R1=(A+B)(C−D)R1=(A+B)(C-D)

Register contents:

```
R1 = (A + B) × (C - D)
```

```
R2 = C - D
R3 = -
R4 = -
```

Therefore:

```
T3 = R1
```

`R2` can now be reused because its value is no longer required.

Instruction 6

```
ADD R1, E
```

Now:

$R1=(A+B)(C-D)+ER1=(A+B)(C-D)+E$

Register contents:

```
R1 = (A+B)(C-D) + E
R2 = C-D
R3 = -
R4 = -
```

Instruction 7

```
MOV X, R1
```

Final result:

```
X = (A+B)(C-D)+E
```

Register contents:

```
R1 = (A+B)(C-D) + E
R2 = C-D
R3 = -
R4 = -
```

Complete Simulation Table

Step	Instruction	R1	R2	R3	R4
0	—	—	—	—	—
1	`MOV R1,A`	A	—	—	—
2	`ADD R1,B`	A+B	—	—	—
3	`MOV R2,C`	A+B	C	—	—
4	`SUB R2,D`	A+B	C-D	—	—
5	`MUL R1,R2`	(A+B)(C-D)	C-D	—	—
6	`ADD R1,E`	(A+B)(C-D)+E	C-D	—	—
7	`MOV X,R1`	(A+B)(C-D)+E	C-D	—	—

v. Efficiency Evaluation

The generated code contains **7 target instructions**.

Number of arithmetic operations

The original expression contains:

- 1 addition: `A+B`
- 1 subtraction: `C-D`
- 1 multiplication
- 1 addition: `+E`

So there are **4 arithmetic operations**.

Memory operations

Only the final result needs to be stored:

```
MOV X, R1
```

The intermediate results `T1`, `T2`, and `T3` remain in registers.

This is more efficient than repeatedly storing temporary values in memory.

Register usage

Only **two registers** are required:

```
R1 → main computation
R2 → second temporary
```

Although four registers are available, only two are necessary.

Q2.A sequence of nested procedure invocations is represented in intermediate form.

Simulation Tasks

i. Simulate runtime stack creation.
ii. Construct activation records.
iii. Allocate storage for parameters and local variables.
iv. Show stack changes during procedure calls and returns.

v. Explain the role of runtime storage management during code generation. (10 Marks)

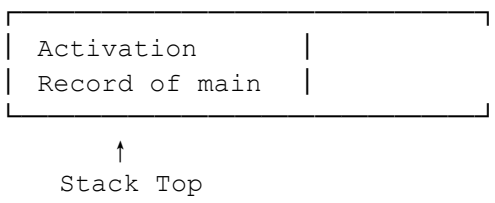
SOLUTION –

i. Simulate Runtime Stack Creation

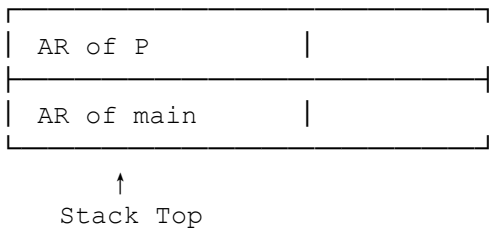
The runtime stack is used to store **activation records** for currently active procedures.

Initially, only `main` is active.

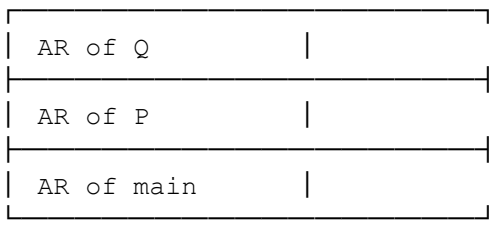
Initial Stack



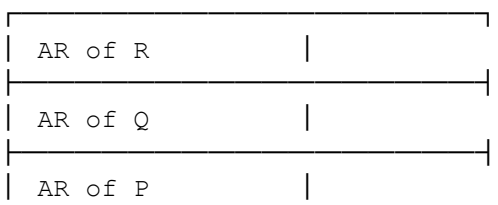
When `main` calls `P(10)`, a new activation record for `P` is pushed.

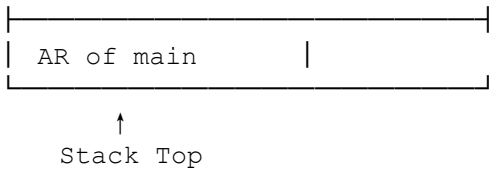


When `P` calls `Q(20)`:



When `Q` calls `R()`:



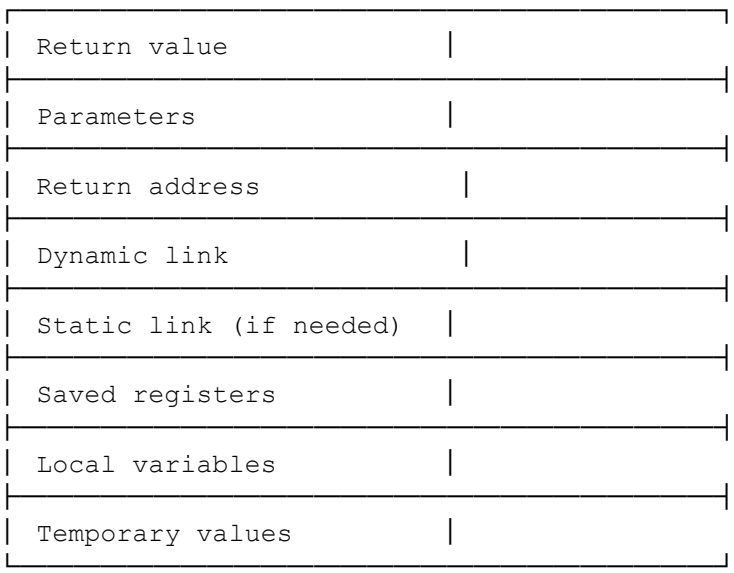


Thus, procedure calls create activation records in **LIFO (Last-In, First-Out)** order.

ii. Construct Activation Records

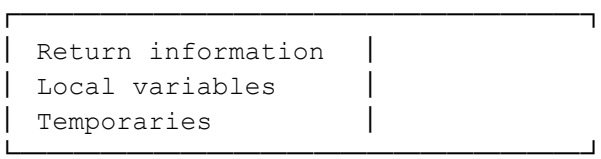
An **activation record (AR)** is a block of memory allocated for one procedure invocation.

A typical activation record contains:

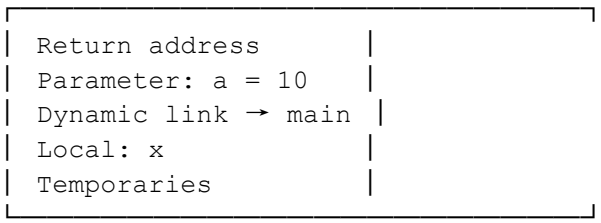


For our example:

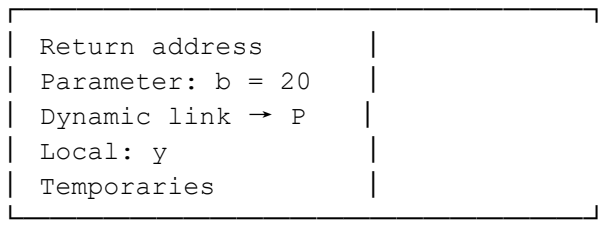
Activation Record of `main`



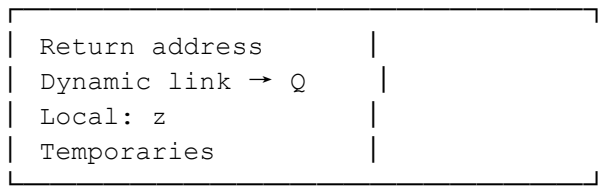
Activation Record of `P(a)`



Activation Record of `Q(b)`



Activation Record of `R()`



---

iii. Allocate Storage for Parameters and Local Variables

Suppose the compiler assigns storage as follows:

Procedure	Parameter	Local Variable	Storage
`main`	—	`m`	AR of main
`P`	`a = 10`	`x`	AR of P
`Q`	`b = 20`	`y`	AR of Q
`R`	—	`z`	AR of R

The important point is that **each procedure invocation gets its own storage**.

For example:

```
P(10)
```

creates storage for:

```
a = 10
x = local storage
```

Then:

```
Q(20)
```

creates a separate storage area:

```
b = 20
y = local storage
```

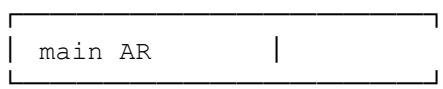
The variables of `P` remain available while `Q` executes.

iv. Show Stack Changes During Calls and Returns

Let's simulate the complete execution.

Step 1 — `main()` starts

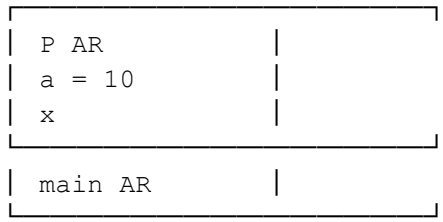
STACK



Step 2 — `main` calls `P(10)`

A new activation record is pushed.

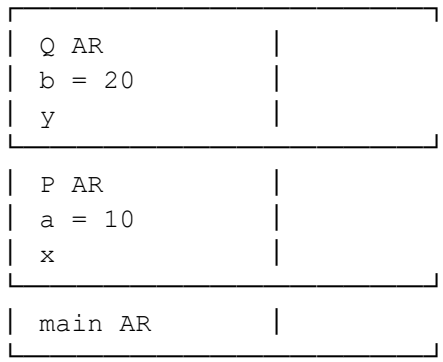
STACK



Stack size increases.

Step 3 — `P` calls `Q(20)`

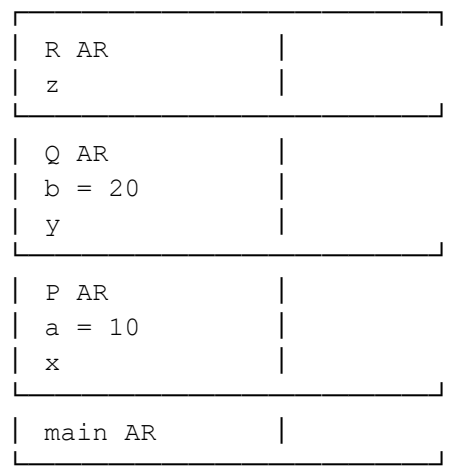
STACK



`P`'s activation record remains on the stack because `P` has not returned.

Step 4 — `Q` calls `R()`

STACK

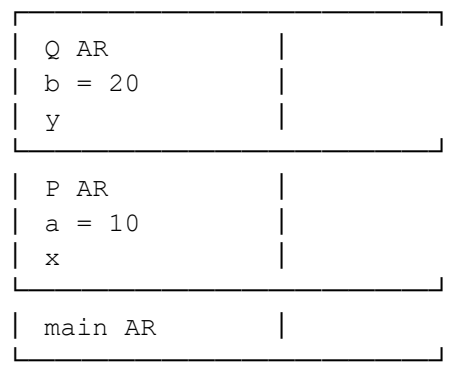


Now four activation records are active.

Step 5 — `R` returns

When `R` finishes, its activation record is removed.

STACK

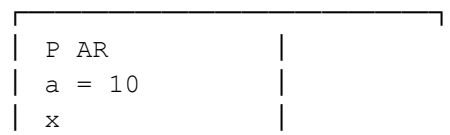


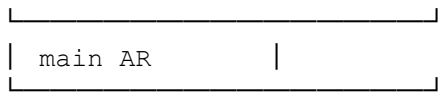
The stack pointer moves back to the previous position.

Step 6 — `Q` returns

`Q`'s activation record is removed.

STACK

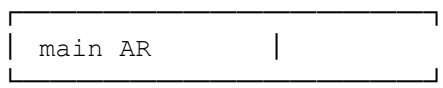




Step 7 — `P` returns

`P`'s activation record is removed.

`STACK`



Step 8 — `main` returns

Finally:

`STACK`

`EMPTY`

The complete stack behavior can therefore be summarized as:

```
main()
   ↓
main
   ↓
main + P
   ↓
main + P + Q
   ↓
main + P + Q + R
   ↓
main + P + Q        ← R returns
   ↓
main + P            ← Q returns
   ↓
main                ← P returns
   ↓
empty               ← main returns
```

v. Role of Runtime Storage Management During Code Generation

Runtime storage management determines **where and how data is stored while the generated program executes**.

The compiler must generate code to manage:

1. Parameters

Parameters must be passed to procedures using registers, stack locations, or other calling-convention mechanisms.

For example:

```
P(10)
```

requires the value `10` to be made available to `P` as parameter `a`.

2. Local Variables

Each procedure needs storage for its local variables.

For example:

```
P:
    local x
```

requires space in `P`'s activation record.

3. Return Address

The return address tells the procedure **where execution should continue after it finishes**.

Conceptually:

```
CALL P
      ↓
P executes
      ↓
RETURN → instruction after CALL P
```

The compiler generates the appropriate call/return instructions.

4. Dynamic and Static Links

A **dynamic link** points to the activation record of the caller.

For example:

```
R → Q → P → main
```

This helps restore the previous execution context when a procedure returns.

A **static link** may be required when nested procedures need to access variables belonging to an enclosing lexical scope.

5. Temporary Storage

Intermediate calculations may require temporary locations.

For example:

```
T1 = a + x
T2 = T1 * 5
```

The compiler must determine whether `T1` and `T2` should be stored in:

- registers,
- stack locations, or
- other temporary storage.

6. Stack Allocation and Deallocation

During a procedure call:

```
CALL P
```

space for `P`'s activation record is allocated.

During:

```
RETURN
```

that space is released.

This allows memory to be reused efficiently.

Q3. Next-use information for variables is available before code generation begins.

Simulation Tasks

i. Construct the next-use table.
ii. Simulate register allocation based on next-use information.
iii. Release registers whenever values become inactive.
iv. Generate the final target instruction sequence.
v. Explain how next-use information improves register utilization. (10 Marks)

SOLUTION –

i. Construct the Next-Use Table

What is next-use information?

The **next use** of a variable tells us the next statement in which that variable will be used.

We calculate it by scanning the intermediate code **backwards**.

For the code:

```
1. T1 = A + B
2. T2 = T1 * C
3. T3 = D + E
4. T4 = T2 - T3
5. X  = T4 + F
```

the next-use information is:

Statement	Variable	Next Use
1	A	Not used again
1	B	Not used again
1	T1	2
2	C	Not used again
2	T2	4
3	D	Not used again
3	E	Not used again
3	T3	4
4	T4	5
5	F	Not used again
5	X	Not used

A more useful table for register allocation is:

Statement	Live values after statement	Next-use information
1	`T1`	T1 → 2
2	`T2`	T2 → 4
3	`T2, T3`	T2 → 4, T3 → 4
4	`T4`	T4 → 5
5	`X`	X → —

The important observation is that after statement 1, **A and B are dead** because they are never used again.

Similarly, after statement 3, **D and E are dead**.

ii. Simulate Register Allocation Using Next-Use Information

We have:

```
R1
R2
```

Statement 1

```
T1 = A + B
```

Load `A` into `R1`:

```
MOV R1, A
```

Then add `B`:

```
ADD R1, B
```

Register status:

```
R1 = T1 = A + B
R2 = FREE
```

Since `A` and `B` have **no future use**, their registers/storage can be considered dead.

Statement 2

```
T2 = T1 * C
```

`T1` is already in `R1`.

```
MUL R1, C
```

Now:

```
R1 = T2 = (A+B) × C
R2 = FREE
```

`T1` is no longer needed because its next use has just occurred.

Therefore:

```
T1 → INACTIVE
```

The register remains occupied by the new value `T2`.

Statement 3

```
T3 = D + E
```

`R2` is free, so allocate `T3` to `R2`.

```
MOV R2, D
ADD R2, E
```

Now:

```
R1 = T2 = (A+B) × C
R2 = T3 = D + E
```

Both values have a future use at statement 4.

Statement 4

```
T4 = T2 - T3
```

Both operands are already in registers:

```
SUB R1, R2
```

Therefore:

$R1=T2-T3R1 = T2-T3$

or:

$R1=((A+B)C)-(D+E)R1=((A+B)C)-(D+E)$

Now `T2` and `T3` are no longer needed.

So:

```
T2 → INACTIVE
T3 → INACTIVE
```

`R2` can now be released.

Register status:

```
R1 = T4
R2 = FREE
```

Statement 5

```
X = T4 + F
```

`T4` is already in `R1`.

```
ADD R1, F
MOV X, R1
```

Final result:

X=((A+B)C)−(D+E)+FX=((A+B)C)-(D+E)+F

iii. Release Registers When Values Become Inactive

The register allocation can be represented as follows:

Step	Instruction	R1	R2	Released
0	—	FREE	FREE	—
1	`MOV R1,A`	A	FREE	—
1	`ADD R1,B`	T1	FREE	A, B
2	`MUL R1,C`	T2	FREE	T1, C
3	`MOV R2,D`	T2	D	—
3	`ADD R2,E`	T2	T3	D, E
4	`SUB R1,R2`	T4	T3	T2, T3
4	—	T4	FREE	R2
5	`ADD R1,F`	X	FREE	T4, F

Notice that **R2 becomes free immediately after statement 4**.

This is an important benefit of next-use analysis.

iv. Final Target Instruction Sequence

The generated target instructions are:

```
1. MOV R1, A
2. ADD R1, B
3. MUL R1, C
4. MOV R2, D
5. ADD R2, E
6. SUB R1, R2
7. ADD R1, F
8. MOV X, R1
```

Register contents after every instruction

Step	Instruction	R1	R2
0	—	FREE	FREE
1	`MOV R1,A`	A	FREE
2	`ADD R1,B`	T1 = A+B	FREE
3	`MUL R1,C`	T2 = (A+B)C	FREE
4	`MOV R2,D`	T2	D
5	`ADD R2,E`	T2	T3 = D+E
6	`SUB R1,R2`	T4 = T2−T3	FREE
7	`ADD R1,F`	X = T4+F	FREE
8	`MOV X,R1`	X	FREE

v. How Next-Use Information Improves Register Utilization

Next-use information helps the compiler determine **which variables should remain in registers and which registers can be safely reused**.

Without next-use information

The compiler may keep old values in registers even when they will never be used again. This causes:

- unnecessary register occupation,
- more register spills,
- additional memory loads/stores,
- slower generated code.

With next-use information

The compiler can identify values that are **dead or inactive**.

For example:

```
T2 = T1 * C
```

After this instruction, `T1` is no longer required. Therefore, the register containing `T1` can immediately be reused for `T2`.

Similarly:

```
T4 = T2 - T3
```

After this instruction, both `T2` and `T3` become inactive. Their registers can be released.

Main advantages

1. **Reduces register spills** to memory.
2. **Reuses registers efficiently.**
3. Keeps frequently needed values in registers.
4. Reduces unnecessary `LOAD` and `STORE` instructions.
5. Produces shorter and faster target code.
6. Helps the compiler choose which value to evict when registers are full.

Q4. An intermediate instruction sequence contains repeated computations inside a loop.

Simulation Tasks

i. **Identify computations repeated during loop execution.**
ii. **Move loop-invariant computations outside the loop.**
iii. **Generate the optimized loop.**
iv. **Compare the execution cost before and after optimization.**
v. **Explain why loop optimization improves performance. (10 Marks)**

SOLUTION –

i. Identify Computations Repeated During Loop Execution

Inside the loop we have:

```
T3 = T1 + i
T4 = T2 * 2
T5 = T3 + T4
```

Now examine each computation.

```
T3 = T1 + i
```

`i` changes every iteration:

```
i = 0, 1, 2, 3, ...
```

Therefore:

```
T1 + i
```

produces a different result each iteration.

 Not loop invariant.

```
T4 = T2 * 2
```

`T2` was calculated before the loop:

```
T2 = C + D
```

Neither `T2` nor `2` changes inside the loop.

Therefore:

```
T2 * 2
```

always produces the same result.

 Loop invariant.

```
T5 = T3 + T4
```

`T3` changes every iteration because it depends on `i`.

Therefore:

```
T3 + T4
```

also changes.

 Not loop invariant.

Repeated Computation

The computation:

```
T4 = T2 * 2
```

is unnecessarily performed **100 times**.

It only needs to be performed **once**.

ii. Move Loop-Invariant Computation Outside the Loop

Original code

```
1. i = 0
2. T1 = A * B
3. T2 = C + D

4. L1:
5.      T3 = T1 + i
6.      T4 = T2 * 2
7.      T5 = T3 + T4
8.      X[i] = T5
9.      i = i + 1
10.     IF i < 100 GOTO L1
```

Move:

```
T4 = T2 * 2
```

before the loop.

iii. Generate the Optimized Loop

The optimized intermediate code becomes:

```
1. i = 0
2. T1 = A * B
3. T2 = C + D
4. T4 = T2 * 2

5. L1:
6.      T3 = T1 + i
7.      T5 = T3 + T4
8.      X[i] = T5
9.      i = i + 1
10.     IF i < 100 GOTO L1
```

Now `T4` is calculated **only once**.

Before optimization

```
T4 = T2 * 2
```

was executed:

100 times100 \text{ times}

After optimization

```
T4 = T2 * 2
```

is executed:

1 time1 \text{ time}

Execution Simulation

Assume:

```
A = 5
B = 4
C = 10
D = 6
```

Then:

```
T1 = A × B
   = 5 × 4
   = 20

T2 = C + D
   = 10 + 6
   = 16
```

The invariant computation is:

```
T4 = T2 × 2
   = 16 × 2
   = 32
```

This is calculated once.

For each iteration:

Iteration 1

```
i = 0

T3 = 20 + 0 = 20
T5 = 20 + 32 = 52
X[0] = 52
```

Iteration 2

```
i = 1

T3 = 20 + 1 = 21
T5 = 21 + 32 = 53
X[1] = 53
```

Iteration 3

```
i = 2

T3 = 20 + 2 = 22
T5 = 22 + 32 = 54
X[2] = 54
```

The value `T4 = 32` remains unchanged.

iv. Compare Execution Cost Before and After Optimization

Assume the loop executes **100 times**.

Let's count arithmetic operations.

Before Optimization

Outside the loop:

```
T1 = A * B      → 1 multiplication
T2 = C + D      → 1 addition
```

Inside the loop:

```
T3 = T1 + i     → 1 addition
T4 = T2 * 2     → 1 multiplication
T5 = T3 + T4    → 1 addition
```

So each iteration requires:

- 2 additions
- 1 multiplication

For 100 iterations:

Multiplications

1+100=1011 + 100 = 101

Additions

1+(100×2)=2011 + (100 \times 2) = 201

Total arithmetic operations:

101+201=302101 + 201 = \boxed{302}

After Optimization

Outside the loop:

```
T1 = A * B      → 1 multiplication
T2 = C + D      → 1 addition
T4 = T2 * 2     → 1 multiplication
```

Inside the loop:

```
T3 = T1 + i     → 1 addition
T5 = T3 + T4    → 1 addition
```

For 100 iterations:

Multiplications

1+1=21+1=2

Additions

$1+(100\times2)=2011+(100\times2)=201$

Total:

$2+201=2032+201=\boxed{203}$

Cost Comparison

Operation	Before Optimization	After Optimization
`A × B`	1	1
`C + D`	1	1
`T2 × 2`	100	**1**
Loop additions	200	200
Total arithmetic operations	**302**	**203**

Therefore:

$302-203=99302-203=\boxed{99}$

arithmetic operations are eliminated.

The multiplication:

```
T4 = T2 * 2
```

is reduced from **100 executions to 1 execution**.

v. Why Loop Optimization Improves Performance

Loop optimization is particularly important because even a small unnecessary computation can become expensive when executed hundreds, thousands, or millions of times.

1. Reduces repeated computation

Instead of calculating:

```
T4 = T2 * 2
```

100 times, we calculate it once.

2. Reduces CPU operations

Fewer arithmetic instructions mean fewer CPU cycles are required.

3. Improves execution speed

The optimized loop contains fewer instructions, so each iteration executes faster.

4. Reduces power consumption

Fewer CPU instructions can also reduce processor activity and energy consumption.

5. Improves register utilization

The invariant value `T4` can be kept in a register and reused throughout the loop rather than being recomputed.

6. Becomes more significant for large loops

If the loop executes NN times, an invariant calculation that costs one operation changes from:

N→1N \rightarrow 1

executions.

For example, with 1,000,000 iterations, that saves:

999,999999,999

executions of the invariant computation.

Q5. A basic block contains multiple repeated arithmetic expressions.

Simulation Tasks

i. **Construct the Directed Acyclic Graph (DAG).**
ii. **Identify common subexpressions.**
iii. **Eliminate redundant computations.**
iv. **Generate optimized intermediate instructions from the DAG.**
v. **Compare the original and optimized instruction sequences. (10 Marks)**

SOLUTION –

i. Construct the DAG

A DAG represents:

- **Leaf nodes** → variables/constants
- **Internal nodes** → operators
- **Edges** → operands
- Identical expressions → **same DAG node**

Step 1: `T1 = A + B`

Create:

Label the `+` node with `T1`.

Step 2: `T2 = C * D`

Label it with `T2`.

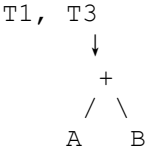
Step 3: `T3 = A + B`

This is exactly the same expression as:

```
T1 = A + B
```

Therefore, **no new node is required**.

The existing `A+B` node gets another label:



Step 4: `T4 = T1 * T2`

Create:

```
  / \
 T1   T2
```

or, using the underlying DAG nodes:

```
    *
   / \
  +   *
 / \ / \
A  B C  D
```

Step 5: `T5 = C * D`

Again:

```
C * D
```

already exists as the node for `T2`.

Therefore:

```
T2, T5
  ↓
   *
  / \
 C   D
```

Step 6: `T6 = T3 + T5`

Since:

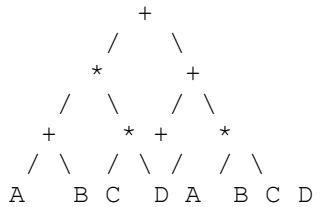
```
T3 = A+B
T5 = C*D
```

this becomes:

```
T6 = (A+B) + (C*D)
```

Create a new + node.

Complete DAG

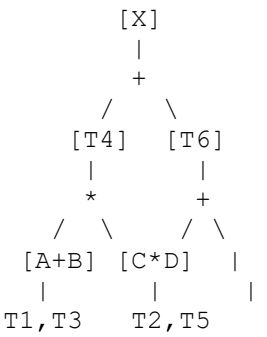


With labels:

```
A + B       → T1, T3
```

```
C * D        → T2, T5
T1 * T2      → T4
T3 + T5      → T6
T4 + T6      → X
```

A cleaner representation is:



ii. Identify Common Subexpressions

Look for expressions that occur more than once.

Common subexpression 1

```
T1 = A + B
T3 = A + B
```

Therefore:

$$A+B\boxed{A+B}$$

is a common subexpression.

Instead of calculating it twice, calculate it once.

Common subexpression 2

```
T2 = C * D
T5 = C * D
```

Therefore:

$$C*D\boxed{C*D}$$

is also a common subexpression.

Instead of calculating it twice, calculate it once.

iii. Eliminate Redundant Computations

Original

```
T1 = A + B
T2 = C * D
T3 = A + B       ← redundant
T4 = T1 * T2
T5 = C * D       ← redundant
T6 = T3 + T5
X  = T4 + T6
```

We eliminate:

```
T3 = A + B
T5 = C * D
```

because the values are already available as `T1` and `T2`.

We can replace:

```
T3 → T1
T5 → T2
```

Thus:

```
T6 = T1 + T2
```

iv. Generate Optimized Intermediate Instructions

Original Intermediate Code

```
1. T1 = A + B
2. T2 = C * D
3. T3 = A + B
4. T4 = T1 * T2
5. T5 = C * D
6. T6 = T3 + T5
7. X  = T4 + T6
```

There are **7 instructions**.

Optimized Intermediate Code

```
1. T1 = A + B
2. T2 = C * D
3. T4 = T1 * T2
4. T6 = T1 + T2
5. X  = T4 + T6
```

There are now only **5 instructions**.

The repeated calculations have been removed.

v. Compare Original and Optimized Instruction Sequences

Step	Original Code	Optimized Code
1	`T1 = A+B`	`T1 = A+B`
2	`T2 = C*D`	`T2 = C*D`
3	`T3 = A+B`	`T4 = T1*T2`
4	`T4 = T1*T2`	`T6 = T1+T2`
5	`T5 = C*D`	`X = T4+T6`
6	`T6 = T3+T5`	—
7	`X = T4+T6`	—

Instruction reduction

Original:

7 instructions7 \text{ instructions}

Optimized:

5 instructions5 \text{ instructions}

Reduction:

7−5=2 instructions7-5=\boxed{2\text{ instructions}}

Percentage reduction:

27×100≈28.57%\frac{2}{7}\times100 \approx \boxed{28.57\%}

More importantly, the number of arithmetic computations is reduced because:

`A + B`

is evaluated **once instead of twice**, and:

`C * D`

is evaluated **once instead of twice**.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation